

SymPy Bug Report

1 Bug 61: Spurious Imaginary Term in Positive-Branch expint Series

1.1 Minimal Reproducer

```
from sympy import EulerGamma, N, Rational, expint, series, symbols

x = symbols("x", positive=True)
s = series(expint(2, x), x, 0, 2)
print("series:", s)

t = Rational(1, 1000)
print("truncated at 1/1000:", N(s.removeO().subs(x, t), 50))
print("actual at 1/1000:", N(expint(2, t), 50))
```

1.2 Observed Behavior

```
series: 1 + x*(log(x) - 1 + EulerGamma - I*pi) + O(x**2)
truncated at 1/1000: 0.99266946038591939580855253772602930980823885487005 -
0.0031415926535897932384626433832795028841971693993751*I
actual at 1/1000: 0.99266896046923884233605257079126963912695444183171
```

SymPy returns an imaginary first-order term, $-i\pi x$, in the small-positive series for $\expint(2, x)$. The numerical comparison at $x = 10^{-3}$ shows the same problem: the truncated series is complex, while direct evaluation of $\expint(2, 10^{-3})$ is real.

1.3 Expected Behavior

For $x > 0$, the expansion at the origin should be

$$\expint(2, x) = 1 + x(\log(x) - 1 + \gamma) + O(x^2),$$

where γ is Euler's constant. No imaginary branch term should appear on the positive real branch.

1.4 Mathematical Explanation

For positive real x , the exponential integral $E_2(x)$ is real and obeys the recurrence

$$E_2(x) = e^{-x} - xE_1(x).$$

The positive-real expansion of E_1 near the origin is

$$E_1(x) = -\gamma - \log(x) + x + O(x^2).$$

Substitution gives

$$\begin{aligned} E_2(x) &= (1 - x + O(x^2)) - x(-\gamma - \log(x) + x + O(x^2)) \\ &= 1 + x(\log(x) - 1 + \gamma) + O(x^2). \end{aligned}$$

The observed term $-i\pi x$ is therefore not an equivalent representation or a harmless branch annotation. It changes the value of the truncated series at positive real inputs from a real number to a complex number.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant code in `sympy/functions/special/error_functions.py`, specifically `Ei._eval_rewrite_as_Si` near lines 1226–1227 and `expint._eval_nseries` near lines 1443–1445. The traced public call is `series(expint(2, x), x, 0, 2)`. For positive integer order $\nu > 1$, `expint._eval_nseries` rewrites through `_eval_rewrite_as_Ei`. In the case $\nu = 2$, this produces an expression equivalent to

$$-x(-\text{Ei}(x \exp_{\text{polar}}(i\pi)) + i\pi) + e^{-x}.$$

The resulting Ei term is then expanded through the Ei n-series method, which rewrites via `Ei._eval_rewrite_as_Si`.

The problematic rule identified in the diagnosis is that `Ei._eval_rewrite_as_Si` uses only `z.is_negative` to decide whether to subtract $i\pi$:

```
if z.is_negative:
    return Shi(z) + Chi(z) - I*pi
```

For $z = x \exp_{\text{polar}}(i\pi)$ with $x > 0$, SymPy reports `z.is_negative == True`. The diagnosis explains that this polar argument is on the upper sheet and already carries the $+i\pi$ logarithmic branch. Treating it like an ordinary negative real argument subtracts an extra $i\pi$, so $\text{Ei}(x \exp_{\text{polar}}(i\pi))$ is expanded as if it were $\gamma + \log(x) - x + O(x^2)$ rather than $\gamma + \log(x) + i\pi - x + O(x^2)$. When this is substituted back into the `expint(2, x)` rewrite, the missing branch constant leaves the spurious $-i\pi x$ term.

A plausible fix direction supported by the diagnosis is to distinguish ordinary negative real arguments from polar negative arguments in `Ei._eval_rewrite_as_Si`. Another supported direction is to avoid this Ei rewrite in `expint._eval_nseries` for positive integer order and instead use a real recurrence or local series for positive real arguments.

1.6 Additional Failing Instances

The independent verification checks the same positive-origin expansion after scaling the positive argument by several positive constants. In each case the expected series follows from the same recurrence applied to $E_2(ax) = e^{-ax} - axE_1(ax)$, so the first-order term should remain real:

$$\text{expint}(2, ax) = 1 + ax(\log(ax) - 1 + \gamma) + O(x^2), \quad a > 0.$$

Input / Expression	SymPy behavior	Expected behavior
<code>series(expint(2, x), x, 0, 2)</code>	Adds $-i\pi x$	$1 + x(\log(x) - 1 + \gamma) + O(x^2)$
<code>series(expint(2, 2x), x, 0, 2)</code>	Adds $-2i\pi x$	$1 + 2x(\log(2x) - 1 + \gamma) + O(x^2)$
<code>series(expint(2, 3x), x, 0, 2)</code>	Adds $-3i\pi x$	$1 + 3x(\log(3x) - 1 + \gamma) + O(x^2)$
<code>series(expint(2, x/2), x, 0, 2)</code>	Adds $-i\pi x/2$	$1 + \frac{x}{2}(\log(x/2) - 1 + \gamma) + O(x^2)$
<code>series(expint(2, 5x), x, 0, 2)</code>	Adds $-5i\pi x$	$1 + 5x(\log(5x) - 1 + \gamma) + O(x^2)$
<code>series(expint(2, 3x/2), x, 0, 2)</code>	Adds $-3i\pi x/2$	$1 + \frac{3x}{2}(\log(3x/2) - 1 + \gamma) + O(x^2)$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public matches in the artifact are SymPy issue #6725, which concerns an expint series problem for fractional order and a negative argument; pull request #26892, which adds nearby expint series and leading-term support; and issue #8712, which discusses polar lifting and branch choices for `expint(1, t exp_polar(i\pi))`. These are related to the broader expint branch/series family, but the available evidence did not identify a clear public duplicate for the exact positive-real small- x case `series(expint(2, x), x, 0, 2)`. The case remains individually actionable because it is a concise, reproducible regression target for a specific branch-term error.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import EulerGamma, O, S, expint, log, series, simplify, symbols

@pytest.mark.parametrize("scale", [S(1), S(2), S(3), S.Half, S(5), 3*S.Half])
def test_expint_two_positive_origin_series_has_no_imaginary_branch_term(scale):
    x = symbols("x", positive=True)
    actual = series(expint(2, scale*x), x, 0, 2)
    expected = 1 + scale*x*(log(scale*x) - 1 + EulerGamma) + O(x**2)
    assert simplify(actual.removeO() - expected.removeO()) == 0
```